

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Rakshitha S	Reg. No.:	192411008
Date:	05.08.2026	Duration:	60 minutes

Code of Conduct

I _____ Rakshitha S / 192411008 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.

Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

Problem statement : AI-Powered Digital Twin Platform for Smart City Infrastructure

Industry Context

Smart cities increasingly adopt digital twin technology to simulate and monitor urban infrastructure in real time. AI, IoT, GIS, and cloud computing enable virtual replicas of roads, utilities, buildings, and transportation systems for predictive maintenance and strategic planning.

Problem Statement

Urban infrastructure management is challenging due to aging assets, increasing population, and limited real-time visibility. Traditional monitoring systems provide fragmented information and lack predictive capabilities. An AI-powered digital twin platform should create virtual models of city infrastructure by integrating real-time IoT data. The system should simulate infrastructure performance, detect anomalies, predict failures, and support data-driven urban planning. This solution will improve infrastructure reliability and maintenance efficiency.

Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning.

Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning decisions.
5. Accurate predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.
8. Sustainable city development.

1. Analyze System Requirements

1.1 System Objectives

The proposed system aims to create an **AI-powered digital twin of smart city infrastructure** by integrating real-time IoT data with AI, GIS, and cloud technologies. The major objectives are:

- Create virtual replicas of roads, utilities, buildings, and transportation systems.
- Continuously collect and process real-time IoT sensor data.
- Detect anomalies and predict infrastructure failures using AI.
- Support predictive maintenance and reduce unexpected failures.
- Simulate urban scenarios for infrastructure planning.
- Provide real-time visualization and performance reports for city authorities.

1.2 Functional Requirements

Functional Requirement	Description
IoT Data Collection	Collect real-time data from sensors installed across city infrastructure.
Digital Twin Creation	Create and continuously update virtual models of physical infrastructure.
AI Analytics	Detect anomalies and predict potential infrastructure failures.
Asset Health Monitoring	Continuously monitor the condition and performance of city assets.
Predictive Maintenance	Generate maintenance alerts before failures occur.
Scenario Simulation	Perform what-if simulations to evaluate infrastructure changes and urban scenarios.
GIS Visualization	Display infrastructure and its condition using spatial/map-based information.
Reporting & Alerts	Generate performance reports, warnings, and maintenance notifications.

1.3 Non-Functional Requirements

- **Scalability:** Must support increasing numbers of sensors, infrastructure assets, and users.
- **Performance:** IoT data and alerts should be processed with minimal delay.
- **Security:** Sensor data, APIs, and user accounts must be protected through authentication, authorization, and encryption.
- **Reliability:** Continuous monitoring and prediction should operate accurately even when individual services fail.
- **Availability:** The platform should remain accessible continuously for city authorities and maintenance teams.
- **Maintainability:** Individual services should be independently modified and upgraded.

These attributes are particularly important because the assessment explicitly expects scalability, maintainability, reliability, availability, performance, and security to be considered during architecture design.

2. Select a Suitable Software Architecture

2.1 Selected Architecture

Hybrid Microservices + Event-Driven Architecture

A **Hybrid Microservices and Event-Driven Architecture** is selected for the proposed digital twin platform.

The system is divided into independent services such as:

- Digital Twin Service
- AI & Analytics Service
- Simulation Service
- GIS Service
- Notification Service
- Data Management Service

The event-driven architecture handles continuous IoT events generated by smart-city infrastructure. A message broker/event bus transfers real-time sensor events to the required services.

2.2 Justification

This architecture is suitable because smart-city infrastructure generates large volumes of continuous real-time data. Event-driven communication enables this information to be processed asynchronously and efficiently.

Microservices allow AI analytics, digital twin modelling, GIS, simulation, and notifications to operate independently. Therefore, heavily used services can be scaled without scaling the entire platform.

The architecture also provides:

- Independent service deployment and maintenance.
- Real-time processing of IoT events.
- Better fault isolation.
- Easy integration with external smart-city systems.
- Flexible addition of new infrastructure and AI models.
- High scalability for city-wide deployment.

3. Software Architecture Diagram

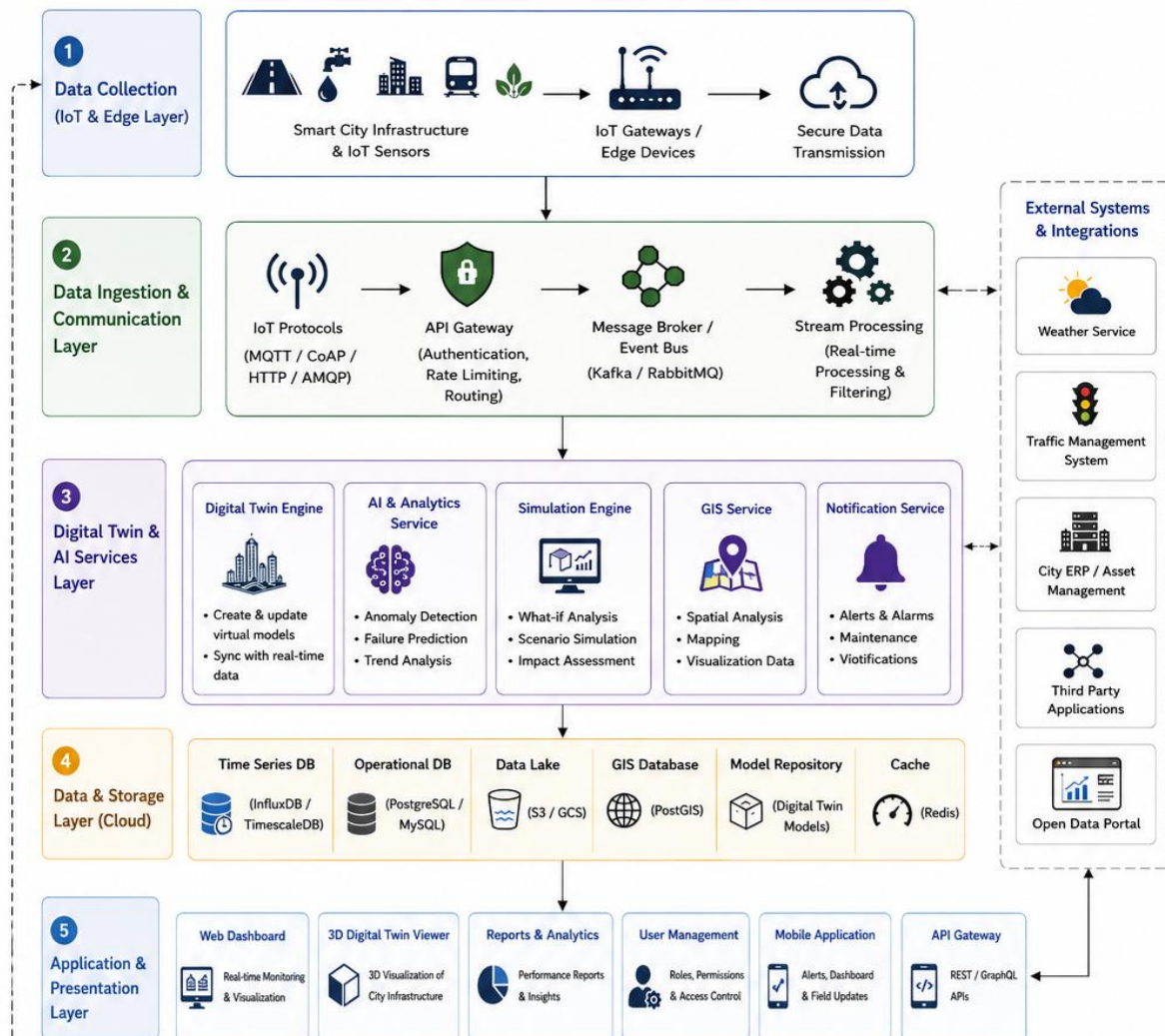


Figure 1: Software Architecture of AI-Powered Digital Twin Platform

The architecture consists of five major layers:

1. Physical Infrastructure & IoT Layer

Contains sensors associated with roads, bridges, buildings, utilities, transportation, and other city infrastructure.

2. Data Ingestion & Communication Layer

IoT gateways collect sensor data and transmit it using protocols such as MQTT/HTTP. The API Gateway, message broker, and stream-processing components manage real-time communication.

3. Digital Twin & AI Services Layer

Contains the Digital Twin Engine, AI Analytics, Simulation, GIS, and Notification services.

4. Data & Storage Layer

Stores sensor data, infrastructure information, GIS data, digital twin models, and historical records using cloud-based databases and storage.

5. Application & Presentation Layer

Provides dashboards, digital twin visualization, reports, alerts, APIs, and mobile access to users.

External weather, traffic, asset-management, and open-data systems can also exchange information with the platform.

4. Components and Their Interactions

4.1 Major Components

Component	Responsibility
IoT Sensors & Gateways	Collect infrastructure condition and environmental data.
API Gateway	Provides secure access, authentication, routing, and API management.
Message Broker	Distributes real-time IoT events between services.
Digital Twin Engine	Creates and synchronizes virtual representations of infrastructure assets.
AI & Analytics Service	Performs anomaly detection, failure prediction, and trend analysis.
Simulation Engine	Performs what-if analysis and infrastructure scenario simulation.
GIS Service	Provides spatial mapping and location-based infrastructure visualization.

Component	Responsibility
Data Storage	Stores sensor readings, asset information, GIS data, and digital twin models.
Notification Service	Generates failure warnings and predictive-maintenance alerts.
Dashboard / 3D Viewer	Displays infrastructure status, predictions, simulations, and digital twins.

4.2 Communication and Data Exchange

IoT sensors continuously transmit infrastructure data to IoT gateways. The data passes through secure communication interfaces and is published to the message broker/event bus.

The Digital Twin Engine uses this data to synchronize virtual infrastructure models, while the AI Analytics Service analyzes current and historical information for anomalies and possible failures.

When a potential problem is identified, the Notification Service generates an alert for infrastructure operators and maintenance teams. The processed information is also presented through dashboards, GIS maps, reports, and the digital twin viewer.

4.3 Overall System Workflow

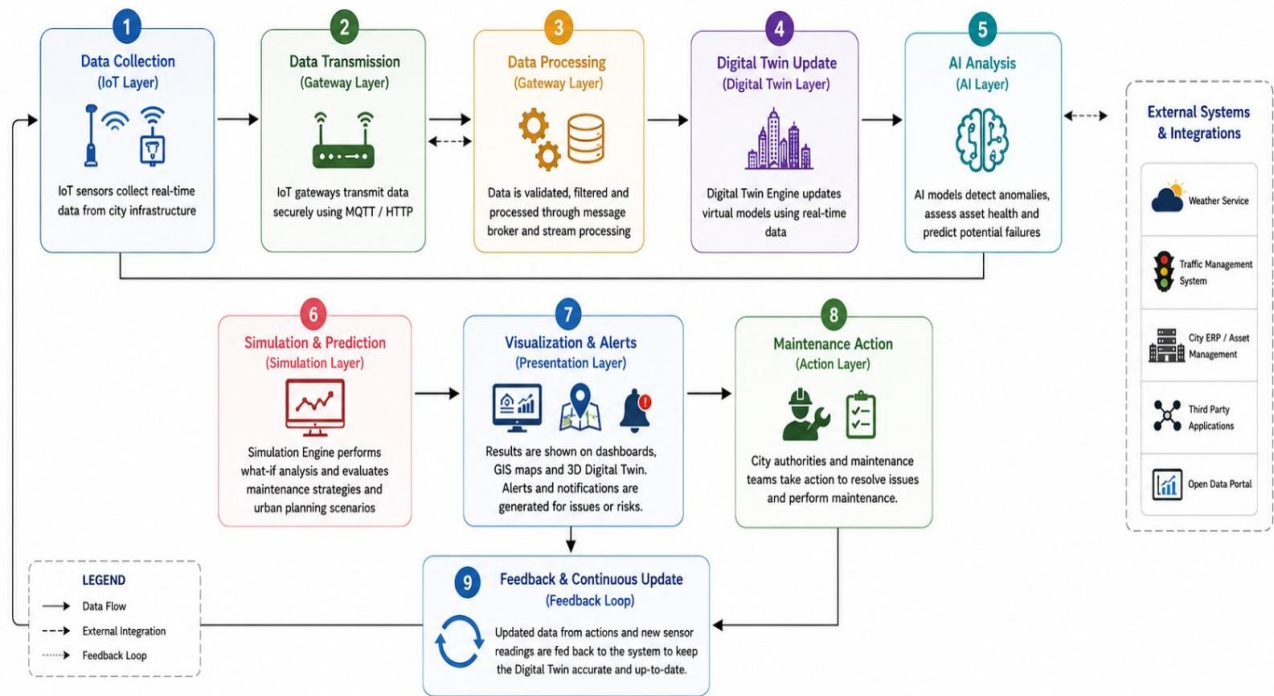


Figure 2: System Workflow of the Digital Twin Platform

The overall workflow can be summarized as:

Physical Infrastructure → IoT Sensors → IoT Gateway → Data Ingestion → Event Processing → Digital Twin Update → AI Analysis → Failure/Anomaly Detection → Simulation & Prediction → Dashboard/Alerts → Maintenance Action → Digital Twin Update

This creates a **continuous feedback loop**, allowing the digital twin to remain synchronized with the physical infrastructure.

5. Quality Attribute Analysis

Quality Attribute	How the Architecture Addresses It
Scalability	Microservices can be independently scaled according to sensor traffic and processing demand.
Security	API authentication, authorization, encrypted communication, and role-based access protect infrastructure data.
Performance	Stream processing and event-driven communication enable rapid processing of real-time sensor information.
Reliability	Independent services provide fault isolation, preventing failure of one component from affecting the entire platform.
Maintainability	Loosely coupled microservices can be updated, tested, and deployed independently.
Availability	Cloud deployment, service replication, and failover mechanisms help maintain continuous system availability.

6. Justify Architectural Decisions

6.1 Rationale

The Hybrid Microservices + Event-Driven Architecture was selected because the platform requires both real-time IoT processing and independently scalable services.

Event-driven communication efficiently handles continuous sensor events, while microservices separate major capabilities such as digital twin management, AI prediction, GIS, simulation, and notifications.

This combination supports the scalability, flexibility, reliability, and real-time performance required for smart-city infrastructure management.

6.2 Architectural Trade-offs

Although the selected architecture offers several benefits, some trade-offs exist:

Advantage	Trade-off
Independent scalability	More deployment complexity
Better fault isolation	Requires service monitoring
Real-time event processing	Message brokers add infrastructure complexity
Independent development	Communication between services must be carefully managed
Easy integration	Strong API and security management is required

The additional complexity is acceptable because the platform requires **large-scale, real-time and highly available infrastructure monitoring**.

6.3 Future Enhancements

The proposed architecture can support future enhancements such as:

- Integration of additional infrastructure such as smart grids and waste-management systems.
- Advanced AI models for more accurate failure prediction.
- Integration with traffic, weather, emergency, and government systems.
- Advanced 3D city-level digital twin visualization.
- Automated maintenance scheduling.
- Expansion from individual infrastructure assets to a complete city-wide digital twin.

Because services are loosely coupled, these features can be introduced without redesigning the entire system.

Conclusion

The proposed AI-Powered Digital Twin Platform combines IoT, AI, GIS, cloud computing, and digital twin technology to continuously monitor and simulate smart-city infrastructure. The selected Hybrid Microservices + Event-Driven Architecture provides the scalability, real-time performance, reliability, maintainability, security, and availability required for city-scale deployment.

By predicting failures, supporting predictive maintenance, and enabling scenario simulation, the platform can improve infrastructure reliability, reduce maintenance costs, and support sustainable smart-city planning.